

Taller de Clase: Regresión con Machine Learning Predicción del precio de vivienda por unidad de área (Real Estate Valuation Dataset)

Integrantes:

Investigador(a) de métricas: Nathalia Mayorga Encargado(a) de EDA y preprocesamiento: Nathalia Mayorga Encargado(a) de modelado: Jeremy contreras Encargado(a) de evaluación y reporte: Jeremy Contreras

Fase 0

0.1 Diferencia entre un problema de regresión y uno de clasificación

La diferencia principal está en el tipo de resultado que predice cada modelo.

Regresión	Clasificación
Predice valores numéricos.	Predice categorías o clases.
Ejemplo: precio de una casa, temperatura o ventas.	Ejemplo: spam/no spam, enfermo/sano.
Se evalúa midiendo el error entre el valor real y el predicho.	Se evalúa verificando si la clasificación fue correcta.

¿Por qué no usan las mismas métricas?

Porque la regresión mide la diferencia entre un valor real y uno predicho, mientras que la clasificación evalúa si la categoría fue correcta o incorrecta. Por ello utilizan métricas diferentes.

0.2 Métricas de evaluación para modelos de regresión

Las métricas permiten medir qué tan precisas son las predicciones de un modelo de regresión. Como cada una analiza el error de manera diferente, es recomendable utilizar varias para obtener una evaluación más completa.

Métrica	¿Qué mide?	¿Cuándo usarla?
MAE	Error absoluto promedio entre el valor real y el predicho.	Cuando todos los errores tienen la misma importancia y no se quiere dar demasiada importancia a los errores grandes.
MSE	Promedio de los errores elevados al cuadrado. Penaliza los errores grandes.	Cuando es importante detectar y castigar errores grandes.
RMSE	Raíz cuadrada del MSE. Mantiene las unidades originales y penaliza errores grandes.	Cuando se desea una métrica fácil de interpretar y sensible a errores grandes.
MAPE	Error promedio expresado en porcentaje.	Cuando se comparan modelos con distintas escalas y no existen valores iguales.
R²	Porcentaje de la variabilidad de los datos explicado por el modelo.	Para conocer qué tan bien el modelo explica los datos.
R² Ajustado	Evalúa el modelo considerando también el número de variables.	Cuando se comparan modelos con diferente cantidad de variables predictoras.

0.3 ¿Por qué el R² puede ser engañoso?

El R² puede aumentar simplemente al agregar más variables al modelo, aunque estas no mejoren realmente las predicciones. Esto puede generar **sobreajuste (overfitting)** y hacer creer que el modelo es mejor de lo que realmente es. Por ello, cuando se comparan modelos con distinta cantidad de variables, es preferible utilizar el **R² ajustado**, ya que penaliza las variables que no aportan información útil.

0.4 Ventaja del RMSE sobre el MAE y del MAE sobre el RMSE

RMSE	MAE
Penaliza mucho los errores grandes, facilitando su detección.	Todos los errores tienen el mismo peso, por lo que es más resistente a los valores atípicos.
Es recomendable cuando un error grande puede tener consecuencias importantes.	Es recomendable cuando se desea conocer el error promedio sin que unos pocos errores extremos influyan demasiado.

Puedes agregar este pequeño párrafo al final de la **pregunta 0.4**:

En conclusión, tanto el RMSE como el MAE son métricas útiles para evaluar modelos de regresión, pero su elección dependerá del objetivo del análisis. Si es importante identificar y penalizar errores grandes, se recomienda utilizar el RMSE; en cambio, si se busca conocer el error promedio de forma más estable y sencilla de interpretar, el MAE es la mejor opción.

```
# =====
# FASE 1: ANÁLISIS EXPLORATORIO DE DATOS
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```

# URL RAW del archivo en GitHub
url_raw = "https://raw.githubusercontent.com/nathaliamayorga718-droid/Real-Estate-Valuation/main/Real%20estate%20valuation9

try:
    df = pd.read_excel(url_raw)
    print("✅ Dataset cargado correctamente desde GitHub")
    print(f"📊 Dimensiones: {df.shape[0]} filas y {df.shape[1]} columnas")
    display(df.head())
except Exception as e:
    print(f"❌ Error al cargar el dataset: {e}")
# =====
# ACTIVIDAD 1: Cargar y revisar datos
# =====
print("="*60)
print("ACTIVIDAD 1: REVISIÓN INICIAL DEL DATASET")
print("="*60)

print(f"\n📌 Dimensiones: {df.shape[0]} filas y {df.shape[1]} columnas")

print("\n📌 Tipos de datos:")
print(df.dtypes)

print("\n📌 Valores nulos por columna:")
print(df.isnull().sum())
print(f"\n📌 Total de valores nulos: {df.isnull().sum().sum()}")

print(f"\n📌 Filas duplicadas: {df.duplicated().sum()}")

# =====
# ACTIVIDAD 2: Estadística descriptiva
# =====
print("\n" + "="*60)
print("ACTIVIDAD 2: ESTADÍSTICA DESCRIPTIVA")
print("="*60)

# Tabla personalizada con media, mediana, desviación estándar y rango
stats = pd.DataFrame({
    'Media': df.mean(numeric_only=True),
    'Mediana': df.median(numeric_only=True),
    'Desv. Estándar': df.std(numeric_only=True),
    'Mínimo': df.min(numeric_only=True),
    'Máximo': df.max(numeric_only=True),
    'Rango': df.max(numeric_only=True) - df.min(numeric_only=True)
})
print("\n📊 Resumen estadístico de cada variable:")
display(stats.round(3))

# =====
# ACTIVIDAD 3: Distribución de la variable objetivo Y
# =====
print("\n" + "="*60)
print("ACTIVIDAD 3: DISTRIBUCIÓN DE LA VARIABLE OBJETIVO (Y)")
print("="*60)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histograma
sns.histplot(df['Y house price of unit area'], bins=25, kde=True,
             color='steelblue', ax=axes[0])
axes[0].axvline(df['Y house price of unit area'].mean(), color='red',
               linestyle='--', label=f'Media: {df["Y house price of unit area"].mean():.2f}')
axes[0].set_title('Histograma del Precio de Vivienda (Y)')
axes[0].set_xlabel('Precio (10,000 NT$/Ping)')
axes[0].legend()

# Boxplot
sns.boxplot(x=df['Y house price of unit area'], color='salmon', ax=axes[1])
axes[1].set_title('Boxplot del Precio (Y) - Detección de Outliers')
axes[1].set_xlabel('Precio')

plt.tight_layout()
plt.show()

# =====
# ACTIVIDAD 4: Matriz de correlación / Heatmap
# =====
print("\n" + "="*60)
print("ACTIVIDAD 4: MATRIZ DE CORRELACIÓN (HEATMAP)")
print("="*60)

plt.figure(figsize=(10, 8))
correlacion = df.corr()

```

```

sns.heatmap(correlacion, annot=True, cmap='coolwarm', fmt=".2f",
             linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('Matriz de Correlación entre Variables')
plt.tight_layout()
plt.show()

# Mostrar las correlaciones más fuertes con Y
print("\n📊 Correlaciones con la variable objetivo (Y):")
corr_y = correlacion['Y house price of unit area'].drop('Y house price of unit area')
for var, valor in corr_y.sort_values(ascending=False).items():
    print(f"    • {var}: {valor:.3f}")

# =====
# ACTIVIDAD 5: Identificación de outliers
# =====
print("\n" + "="*60)
print("ACTIVIDAD 5: IDENTIFICACIÓN DE OUTLIERS")
print("="*60)

variables = ['X2 house age',
             'X3 distance to the nearest MRT station',
             'X4 number of convenience stores',
             'Y house price of unit area']

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, var in enumerate(variables):
    sns.boxplot(x=df[var], color='lightgreen', ax=axes[i])
    axes[i].set_title(f'Boxplot de {var}')

plt.tight_layout()
plt.show()

# Contar outliers usando el método IQR: Rango Inter cuartilico.
# Es un método estadístico para detectar valores muy alejados del resto
print("\n📊 Análisis de outliers por variable (método IQR):")
for var in variables:
    Q1 = df[var].quantile(0.25)
    Q3 = df[var].quantile(0.75)
    IQR = Q3 - Q1
    inferior = Q1 - 1.5 * IQR
    superior = Q3 + 1.5 * IQR
    outliers = df[(df[var] < inferior) | (df[var] > superior)]
    print(f"    • {var}: {len(outliers)} outliers ({len(outliers)/len(df)*100:.1f}%)")

print("\n✅ FASE 1 COMPLETADA")

```


✓ Dataset cargado correctamente desde GitHub
Dimensiones: 414 filas y 8 columnas

	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	1	2012.916667	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.916667	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583333	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500000	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833333	5.0	390.56840	5	24.97937	121.54245	43.1

=====

ACTIVIDAD 1: REVISIÓN INICIAL DEL DATASET

=====

✦ Dimensiones: 414 filas y 8 columnas

✦ Tipos de datos:

```
No int64
X1 transaction date float64
X2 house age float64
X3 distance to the nearest MRT station float64
X4 number of convenience stores int64
X5 latitude float64
X6 longitude float64
Y house price of unit area float64
dtype: object
```

✦ Valores nulos por columna:

```
No 0
X1 transaction date 0
X2 house age 0
X3 distance to the nearest MRT station 0
X4 number of convenience stores 0
X5 latitude 0
X6 longitude 0
Y house price of unit area 0
dtype: int64
```

✦ Total de valores nulos: 0

✦ Filas duplicadas: 0

=====

ACTIVIDAD 2: ESTADÍSTICA DESCRIPTIVA

=====

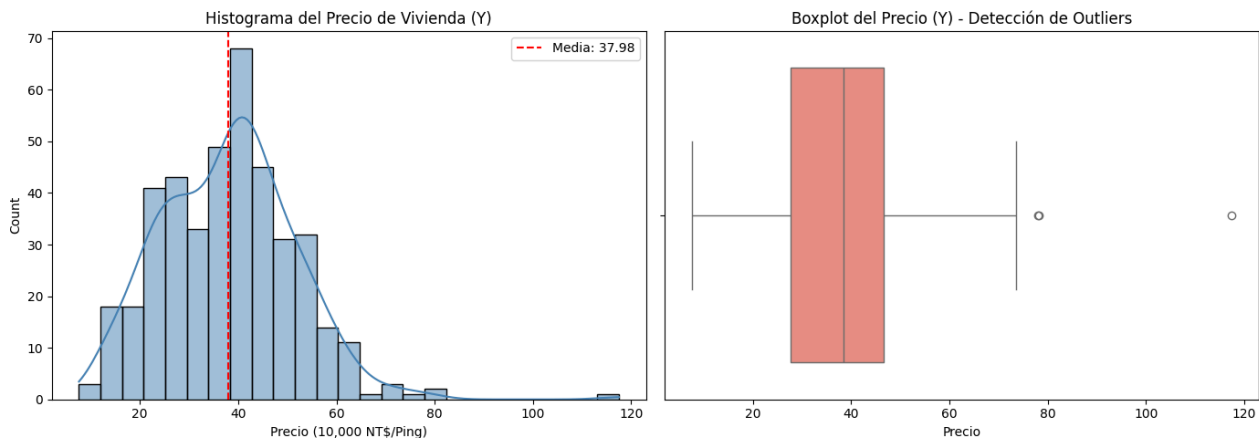
📊 Resumen estadístico de cada variable:

	Media	Mediana	Desv. Estándar	Mínimo	Máximo	Rango
No	207.500	207.500	119.656	1.000	414.000	413.000
X1 transaction date	2013.149	2013.167	0.282	2012.667	2013.583	0.917
X2 house age	17.713	16.100	11.392	0.000	43.800	43.800
X3 distance to the nearest MRT station	1083.886	492.231	1262.110	23.383	6488.021	6464.638
X4 number of convenience stores	4.094	4.000	2.946	0.000	10.000	10.000
X5 latitude	24.969	24.971	0.012	24.932	25.015	0.083
X6 longitude	121.533	121.539	0.015	121.474	121.566	0.093
Y house price of unit area	37.980	38.450	13.606	7.600	117.500	109.900

=====

ACTIVIDAD 3: DISTRIBUCIÓN DE LA VARIABLE OBJETIVO (Y)

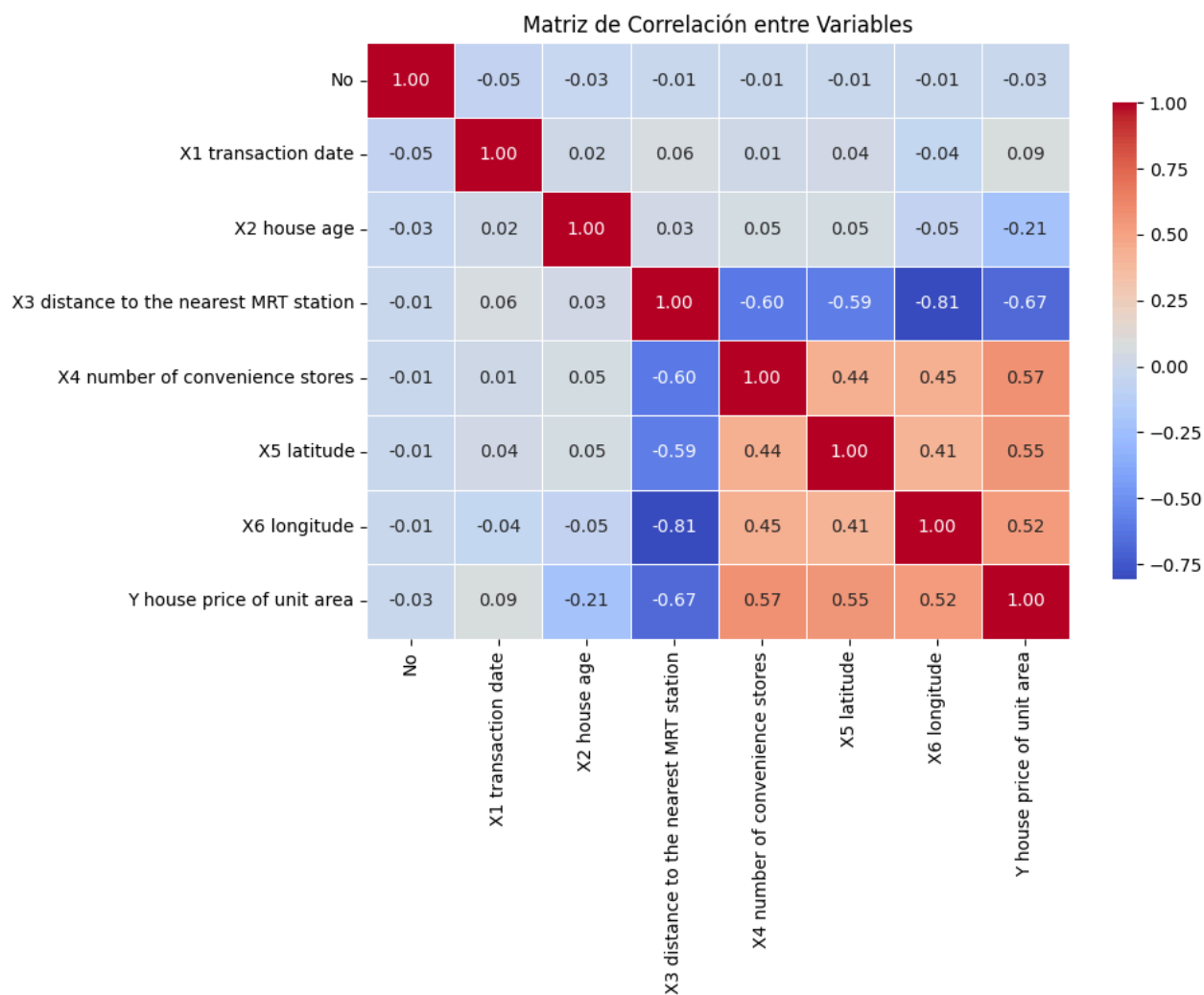
=====



=====

ACTIVIDAD 4: MATRIZ DE CORRELACIÓN (HEATMAP)

=====



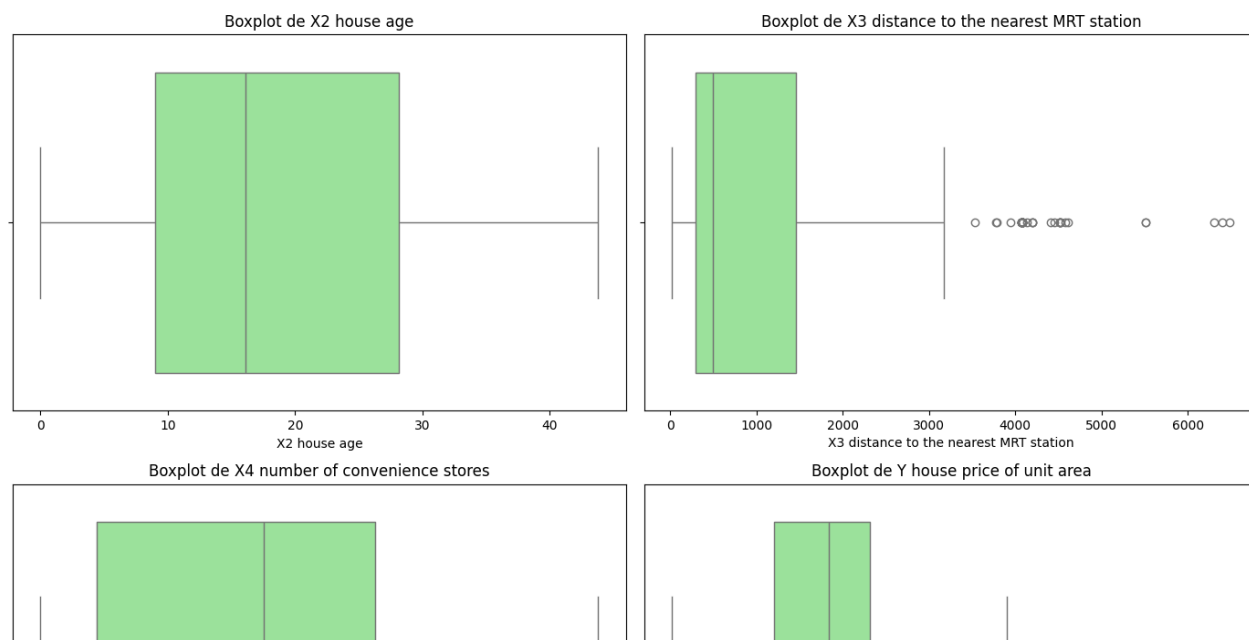
Correlaciones con la variable objetivo (Y):

- X4 number of convenience stores: 0.571
- X5 latitude: 0.546
- X6 longitude: 0.523
- X1 transaction date: 0.088
- No: -0.029
- X2 house age: -0.211
- X3 distance to the nearest MRT station: -0.674

=====

ACTIVIDAD 5: IDENTIFICACIÓN DE OUTLIERS

=====



En la Fase 1 (Análisis Exploratorio de Datos), se realizó una inspección exhaustiva del dataset, confirmando su calidad inicial al no presentar valores nulos ni duplicados. Mediante estadísticos descriptivos y visualizaciones, se analizó el comportamiento de la variable objetivo, identificando un outlier extremo (un precio de 117.5) que distorsionaba la distribución general. Asimismo, la matriz de correlación reveló que la cercanía al MRT y el número de tiendas cercanas son los principales impulsores del precio, mientras que se detectó una alta multicolinealidad entre las coordenadas geográficas (latitud y longitud).

```
# =====
# FASE 2: PREPROCESAMIENTO DE DATOS
# =====

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

# =====
# ACTIVIDAD 1: Eliminar la columna No (ID)
# =====
print("\n" + "="*60)
print("ACTIVIDAD 1: ELIMINAR COLUMNA ID (No)")
print("\n" + "="*60)

print(f"\n 🚩 Columnas antes: {df.columns.tolist()}")
df = df.drop(columns=['No'])
print(f" 🚩 Columnas después: {df.columns.tolist()}")
print("✅ Columna 'No' eliminada (es solo un identificador, no aporta información)")

# =====
# ACTIVIDAD 2: Tratamiento de outliers
# =====
print("\n" + "="*60)
print("ACTIVIDAD 2: TRATAMIENTO DE OUTLIERS")
print("\n" + "="*60)

# 2.1. Eliminar el outlier extremo en Y (precio = 117.5)
print("\n 🚩 2.1. Eliminando outlier extremo en Y (precio > 100)...")
print(f" • Filas antes: {len(df)}")
print(f" • Valor máximo de Y antes: {df['Y house price of unit area'].max()}")

df = df[df['Y house price of unit area'] < 100]

print(f" • Filas después: {len(df)}")
print(f" • Valor máximo de Y después: {df['Y house price of unit area'].max()}")
print("✅ Outlier extremo eliminado")

# 2.2. Transformar X3 (Distancia al MRT) con logaritmo
print("\n 🚩 2.2. Aplicando transformación logarítmica a X3 (Distancia al MRT)...")
print(f" • Rango original: [{df['X3 distance to the nearest MRT station'].min():.2f}, {df['X3 distance to the nearest MRT station'].max():.2f}]")

df['X3 distance to the nearest MRT station'] = np.log1p(df['X3 distance to the nearest MRT station'])

print(f" • Rango después de log1p: [{df['X3 distance to the nearest MRT station'].min():.2f}, {df['X3 distance to the nearest MRT station'].max():.2f}]")
print("✅ Transformación logarítmica aplicada")

# Verificar visualmente los cambios
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Boxplot de Y antes y después (simulado con el dataset limpio)
sns.boxplot(x=df['Y house price of unit area'], color='lightgreen', ax=axes[0])
axes[0].set_title('Boxplot de Y (después de eliminar outlier)')
axes[0].set_xlabel('Precio')

# Histograma de X3 transformada
sns.histplot(df['X3 distance to the nearest MRT station'], bins=30, kde=True, color='orange', ax=axes[1])
axes[1].set_title('Distribución de X3 (después de log1p)')
axes[1].set_xlabel('Distancia al MRT (log)')

plt.tight_layout()
plt.show()

# =====
# ACTIVIDAD 3: División Train/Test (80/20)
# =====
print("\n" + "="*60)
print("ACTIVIDAD 3: DIVISIÓN TRAIN/TEST (80/20)")
print("\n" + "="*60)

# Separar features (X) y target (y)
```

```

X = df.drop(columns=['Y house price of unit area'])
y = df['Y house price of unit area']

print(f"\n🔴 Shape de X (features): {X.shape}")
print(f"🔴 Shape de y (target): {y.shape}")

# Dividir en 80% entrenamiento y 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% para prueba
    random_state=42,        # Semilla fija para reproducibilidad
    shuffle=True            # Mezclar los datos antes de dividir
)

print(f"\n✅ División completada:")
print(f"    • Train (entrenamiento): {X_train.shape[0]} muestras")
print(f"    • Test (prueba): {X_test.shape[0]} muestras")
print(f"    • Semilla aleatoria (random_state): 42")

# =====
# ACTIVIDAD 4: Escalado de variables (StandardScaler)
# =====
print("\n" + "="*60)
print("ACTIVIDAD 4: ESCALADO DE VARIABLES (STANDARDSCALER)")
print("="*60)

print("\n💡 ¿Por qué escalamos?")
print("    • Algunas variables tienen rangos muy diferentes (ej: X3 va de 23 a 6488)")
print("    • Sin escalar, el modelo le daría más importancia a las variables con números grandes")
print("    • StandardScaler convierte todas las variables a media=0 y desviación=1")

# Crear el escalador
scaler = StandardScaler()

# Ajustar el escalador SOLO con los datos de entrenamiento (evita data leakage)
X_train_scaled = scaler.fit_transform(X_train)

# Aplicar la misma transformación a los datos de prueba
X_test_scaled = scaler.transform(X_test)

# Convertir de nuevo a DataFrame para mantener los nombres de columnas
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns)

print("\n📊 Comparativa ANTES vs DESPUÉS del escalado (datos de entrenamiento):")
print("\n--- ANTES (X_train original) ---")
print(X_train.describe().loc[['mean', 'std']].round(3))

print("\n--- DESPUÉS (X_train escalado) ---")
print(X_train_scaled.describe().loc[['mean', 'std']].round(3))

print("\n✅ Escalado completado")
print("    • Todas las variables ahora tienen media ≈ 0 y desviación estándar ≈ 1")

# =====
# RESUMEN FINAL DE LA FASE 2
# =====
print("\n" + "="*60)
print("✅ FASE 2 COMPLETADA - RESUMEN")
print("="*60)
print(f"🔴 Dataset final: {len(df)} registros limpios")
print(f"🔴 Features utilizadas: {X.shape[1]}")
print(f"🔴 Transformaciones aplicadas:")
print(f"    • Eliminada columna 'No' (ID)")
print(f"    • Eliminado 1 outlier extremo en Y (precio = 117.5)")
print(f"    • Distancia al MRT (X3) transformada con log1p")
print(f"🔴 División: Train = {len(X_train)} | Test = {len(X_test)}")
print(f"🔴 Escalado: StandardScaler aplicado")
print(f"\n🎉 Los datos están listos para la FASE 3: MODELADO")

```

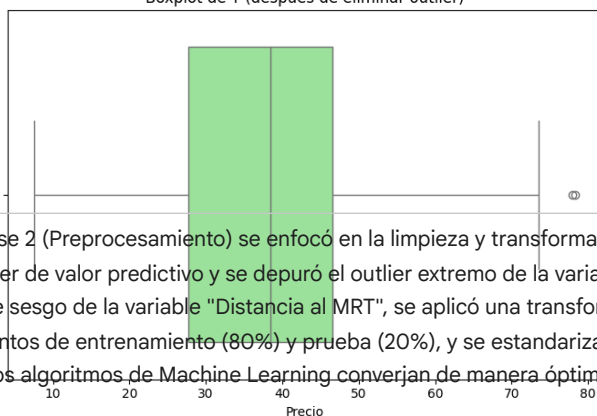

ACTIVIDAD 1: ELIMINAR COLUMNA ID (No)

- ✦ Columnas antes: ['No', 'X1 transaction date', 'X2 house age', 'X3 distance to the nearest MRT station', 'X4 number of co
- ✦ Columnas después: ['X1 transaction date', 'X2 house age', 'X3 distance to the nearest MRT station', 'X4 number of conven
- ✓ Columna 'No' eliminada (es solo un identificador, no aporta información)

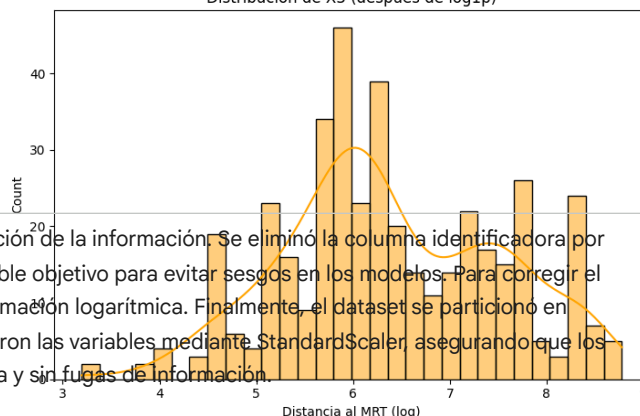
ACTIVIDAD 2: TRATAMIENTO DE OUTLIERS

- ♦ 2.1. Eliminando outlier extremo en Y (precio > 100)...
 - Filas antes: 414
 - Valor máximo de Y antes: 117.5
 - Filas después: 413
 - Valor máximo de Y después: 78.3
- ✓ Outlier extremo eliminado
- ♦ 2.2. Aplicando transformación logarítmica a X3 (Distancia al MRT)...
 - Rango original: [23.38, 6488.02]
 - Rango después de log1p: [3.19, 8.78]
- ✓ Transformación logarítmica aplicada

Boxplot de Y (después de eliminar outlier)



Distribución de X3 (después de log1p)



La Fase 2 (Preprocesamiento) se enfocó en la limpieza y transformación de la información. Se eliminó la columna identificadora por carecer de valor predictivo y se depuró el outlier extremo de la variable objetivo para evitar sesgos en los modelos. Para corregir el fuerte sesgo de la variable "Distancia al MRT", se aplicó una transformación logarítmica. Finalmente, el dataset se particionó en conjuntos de entrenamiento (80%) y prueba (20%), y se estandarizaron las variables mediante StandardScaler, asegurando que los futuros algoritmos de Machine Learning converjan de manera óptima y sin fugas de información.

```
#####
# FASE 3: MODELADO Y EVALUACIÓN DE MODELOS
#####
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

warnings.filterwarnings('ignore')

print("="*60)
print("✦ FASE 3: MODELADO DE DATOS")
print("="*60)

# NOTA: Por si no guardaste el escalado al final de la Fase 2,
# lo volvemos a hacer aquí para asegurar que el código corra perfecto.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Diccionario para guardar los resultados y compararlos al final
resultados = {}

# Función auxiliar para evaluar y no repetir código
def evaluar_modelo(nombre, y_real, y_pred):
    mse = mean_squared_error(y_real, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_real, y_pred)
    r2 = r2_score(y_real, y_pred)

    print(f"\n📊 Métricas para {nombre}:")
    print(f"• MAE (Error Absoluto Medio): {mae:.4f}")
    print(f"• RMSE (Raíz del Error Cuadrático Medio): {rmse:.4f}")
    print(f"• R2 (Coeficiente de Determinación): {r2:.4f}")

    resultados[nombre] = {'RMSE': rmse, 'R2': r2}
    return y_pred

#####
# MODELO 1: Regresión Lineal Múltiple
#####
print("\n" + "="*60)
print("♦ MODELO 1: REGRESIÓN LINEAL MÚLTIPLE")
```

```

print("="*60)

# Los modelos lineales SÍ requieren datos escalados
modelo_lr = LinearRegression()
modelo_lr.fit(X_train_scaled, y_train)
y_pred_lr = modelo_lr.predict(X_test_scaled)

y_pred_lr = evaluar_modelo("Regresión Lineal", y_test, y_pred_lr)
print("✅ Modelo Lineal entrenado y evaluado.")

#=====
# MODELO 2: Random Forest Regressor
#=====
print("\n" + "="*60)
print("💡 MODELO 2: RANDOM FOREST REGRESSOR")
print("="*60)

# Los árboles de decisión NO requieren datos escalados
modelo_rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
modelo_rf.fit(X_train, y_train)
y_pred_rf = modelo_rf.predict(X_test)

y_pred_rf = evaluar_modelo("Random Forest", y_test, y_pred_rf)
print("✅ Random Forest entrenado y evaluado.")

#=====
# MODELO 3: Gradient Boosting Regressor
#=====
print("\n" + "="*60)
print("💡 MODELO 3: GRADIENT BOOSTING REGRESSOR")
print("="*60)

modelo_gb = GradientBoostingRegressor(n_estimators=100, random_state=42)
modelo_gb.fit(X_train, y_train)
y_pred_gb = modelo_gb.predict(X_test)

y_pred_gb = evaluar_modelo("Gradient Boosting", y_test, y_pred_gb)
print("✅ Gradient Boosting entrenado y evaluado.")

#=====
# ACTIVIDAD EXTRA: Ajuste de Hiperparámetros (Random Forest)
#=====
print("\n" + "="*60)
print("🔍 OPTIMIZACIÓN: RANDOMIZED SEARCH CV (RANDOM FOREST)")
print("="*60)

# Definimos la cuadrícula de hiperparámetros a explorar
param_distribucion = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

print("🔍 Buscando los mejores hiperparámetros (esto puede tardar unos segundos)...")
random_search = RandomizedSearchCV(
    estimator=modelo_rf,
    param_distributions=param_distribucion,
    n_iter=10,          # Número de combinaciones aleatorias a probar
    cv=3,              # Validación cruzada de 3 folds
    verbose=1,
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)

print(f"\n🏆 Mejores hiperparámetros encontrados: {random_search.best_params_}")
print(f"📊 Mejor R2 en validación cruzada: {random_search.best_score_:.4f}")

# Evaluamos el modelo optimizado
mejor_rf = random_search.best_estimator_
y_pred_mejor_rf = mejor_rf.predict(X_test)
y_pred_mejor_rf = evaluar_modelo("Random Forest (Optimizado)", y_test, y_pred_mejor_rf)
print("✅ Modelo optimizado evaluado.")

#=====
# VISUALIZACIÓN 1: Valores Reales vs Predichos
#=====
print("\n" + "="*60)
print("📊 GRÁFICO 1: REALES VS PREDICHOS")
print("="*60)

```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Comparación: Precio Real vs Precio Predicho por Modelo', fontsize=16, fontweight='bold')

modelos_preds = {
    'Regresión Lineal': y_pred_lr,
    'Random Forest': y_pred_rf,
    'Gradient Boosting': y_pred_gb
}

for ax, (nombre, preds) in zip(axes, modelos_preds.items()):
    sns.scatterplot(x=y_test, y=preds, ax=ax, alpha=0.6, color='teal')
    ax.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Predicción Perfecta')
    ax.set_title(nombre)
    ax.set_xlabel('Precio Real')
    ax.set_ylabel('Precio Predicho')
    ax.legend()

plt.tight_layout()
plt.show()

=====
# VISUALIZACIÓN 2: Importancia de Variables (Modelos de Árbol)
=====
print("\n" + "="*60)
print(" 🌳 GRÁFICO 2: IMPORTANCIA DE VARIABLES")
print("="*60)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Random Forest
importancias_rf = pd.DataFrame({
    'Variable': X_train.columns,
    'Importancia': mejor_rf.feature_importances_
}).sort_values(by='Importancia', ascending=False)

sns.barplot(x='Importancia', y='Variable', data=importancias_rf, ax=axes[0], palette='viridis')
axes[0].set_title('Importancia de Variables (Random Forest Optimizado)')

# Gradient Boosting
importancias_gb = pd.DataFrame({
    'Variable': X_train.columns,
    'Importancia': modelo_gb.feature_importances_
}).sort_values(by='Importancia', ascending=False)

sns.barplot(x='Importancia', y='Variable', data=importancias_gb, ax=axes[1], palette='magma')
axes[1].set_title('Importancia de Variables (Gradient Boosting)')

plt.tight_layout()
plt.show()

=====
# VISUALIZACIÓN 3: Comparativa de Rendimiento (R2 y RMSE)
=====
print("\n" + "="*60)
print(" 📊 GRÁFICO 3: COMPARATIVA FINAL DE MODELOS")
print("="*60)

df_resultados = pd.DataFrame(resultados).T
df_resultados['Modelo'] = df_resultados.index

fig, ax1 = plt.subplots(figsize=(10, 6))

# Gráfico de barras para R2
sns.barplot(x='Modelo', y='R2', data=df_resultados, palette='coolwarm', ax=ax1)
ax1.set_ylabel('R2 Score (Mayor es mejor)', color='teal')
ax1.set_ylim(0, 1)
ax1.tick_params(axis='y', labelcolor='teal')
ax1.set_title('Rendimiento Comparativo de los Modelos de Regresión', fontweight='bold')

# Agregar los valores de R2 encima de las barras
for i, v in enumerate(df_resultados['R2']):
    ax1.text(i, v + 0.02, f"{v:.3f}", ha='center', fontweight='bold', color='black')

plt.tight_layout()
plt.show()

print("\n✅ FASE 3 COMPLETADA")

```


FASE 3: MODELADO DE DATOS

MODELO 1: REGRESIÓN LINEAL MÚLTIPLE

Métricas para Regresión Lineal:

- MAE (Error Absoluto Medio): 5.1951
- RMSE (Raíz del Error Cuadrático Medio): 8.2056
- R2 (Coeficiente de Determinación): 0.5649

Modelo Lineal entrenado y evaluado.

MODELO 2: RANDOM FOREST REGRESSOR

Métricas para Random Forest:

- MAE (Error Absoluto Medio): 4.7882
- RMSE (Raíz del Error Cuadrático Medio): 7.1959
- R2 (Coeficiente de Determinación): 0.6654

Random Forest entrenado y evaluado.

MODELO 3: GRADIENT BOOSTING REGRESSOR

Métricas para Gradient Boosting:

- MAE (Error Absoluto Medio): 4.7040
- RMSE (Raíz del Error Cuadrático Medio): 7.0749
- R2 (Coeficiente de Determinación): 0.6765

Gradient Boosting entrenado y evaluado.

OPTIMIZACIÓN: RANDOMIZED SEARCH CV (RANDOM FOREST)

Buscando los mejores hiperparámetros (esto puede tardar unos segundos)...
Fitting 3 folds for each of 10 candidates, totalling 30 fits

Mejores hiperparámetros encontrados: {'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_depth': 1}
Mejor R2 en validación cruzada: 0.7640

Métricas para Random Forest (Optimizado):

- MAE (Error Absoluto Medio): 4.7140
- RMSE (Raíz del Error Cuadrático Medio): 6.9855
- R2 (Coeficiente de Determinación): 0.6846

Modelo optimizado evaluado.

GRÁFICO 1: REALES VS PREDICHOS

Comparación: Precio Real vs Precio Predicho por Modelo



La Fase 3 Modelado se enfocó en entrenar tres algoritmos de Machine Learning para predecir el precio de las viviendas: Regresión Lineal Múltiple, Random Forest y Gradient Boosting. La Regresión Lineal se entrenó con datos estandarizados, mientras que los modelos basados en árboles utilizaron los datos originales. Además, se optimizó el modelo Random Forest mediante RandomizedSearchCV con validación cruzada para mejorar su rendimiento. Los resultados mostraron que los modelos de ensamble obtuvieron un mejor desempeño, alcanzando el Random Forest Optimizado un R^2 de 0.685, frente a la Regresión Lineal que obtuvo un R^2 de 0.565, evidenciando relaciones no lineales entre las variables y el precio de las viviendas. Finalmente, se generaron las predicciones para su evaluación comparativa.

Importancia de Variables (Random Forest Optimizado)

Importancia de Variables (Gradient Boosting)

```
#####
# FASE 4: EVALUACIÓN Y COMPARACIÓN DE MODELOS
#####
from sklearn.metrics import mean_absolute_percentage_error
# BLOQUE DE SEGURIDAD (Ejecuta esto solo si reiniciaste el entorno y perdiste las variables)
try:
    _ = X_train.shape
except NameError:
    print("⚠ Variables no encontradas. Cargando y preparando datos automáticamente...")
    from sklearn.model_selection import train_test_split, RandomizedSearchCV
    from sklearn.preprocessing import StandardScaler
    from sklearn.linear_model import LinearRegression
    from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
```

```

url_raw = "https://archive.ics.uci.edu/ml/machine-learning-databases/00477/RealEstate.csv"
df = pd.read_csv(url_raw)
df.columns = ['No', 'X1 transaction date', 'X2 house age', 'X3 distance to the nearest MRT station',
              'X4 number of convenience stores', 'X5 latitude', 'X6 longitude', 'Y house price of unit area']

df = df.drop(columns=['No'])
df = df[df['Y house price of unit area'] < 100]
df['X3 distance to the nearest MRT station'] = np.log1p(df['X3 distance to the nearest MRT station'])
X = df.drop(columns=['Y house price of unit area'])
y = df['Y house price of unit area']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
modelo_lr = LinearRegression().fit(X_train_scaled, y_train)
mejor_rf = RandomizedSearchCV(RandomForestRegressor(random_state=42), {'n_estimators':[100,200]}, n_iter=1, cv=2, n_jobs=-1)
modelo_gb = GradientBoostingRegressor(random_state=42).fit(X_train_scaled, y_train)
print("✅ Entorno restaurado correctamente.")

print("="*60)
print("📊 FASE 4: EVALUACIÓN Y COMPARACIÓN DE MODELOS")
print("="*60)

# Función para calcular todas las métricas pedidas
def calcular_metricas_completas(y_real, y_pred, n_features, nombre_modelo):
    n = len(y_real)
    p = n_features

    mae = mean_absolute_error(y_real, y_pred)
    mse = mean_squared_error(y_real, y_pred)
    rmse = np.sqrt(mse)
    mape = mean_absolute_percentage_error(y_real, y_pred) * 100 # En porcentaje
    r2 = r2_score(y_real, y_pred)
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - p - 1) # Fórmula del R2 ajustado

    print(f"\n📊 Métricas para: {nombre_modelo}")
    print(f" • MAE (Error Absoluto Medio): {mae:.4f}")
    print(f" • MSE (Error Cuadrático Medio): {mse:.4f}")
    print(f" • RMSE (Raíz del Error Cuadrático): {rmse:.4f}")
    print(f" • MAPE (Error Porcentual Medio): {mape:.2f}%")
    print(f" • R² (Coeficiente de Determinación): {r2:.4f}")
    print(f" • R² Ajustado: {r2_adj:.4f}")

    return {'MAE': mae, 'MSE': mse, 'RMSE': rmse, 'MAPE (%)': mape, 'R2': r2, 'R2 Ajustado': r2_adj}

#=====
# ACTIVIDAD 1: Calcular métricas y Tabla Comparativa
#=====
print("\n" + "="*60)
print("💡 CALCULANDO MÉTRICAS...")
print("="*60)

preds_lr = modelo_lr.predict(X_test_scaled)
preds_rf = mejor_rf.predict(X_test)
preds_gb = modelo_gb.predict(X_test)

n_features = X_train.shape[1]

metricas_lr = calcular_metricas_completas(y_test, preds_lr, n_features, "Regresión Lineal")
metricas_rf = calcular_metricas_completas(y_test, preds_rf, n_features, "Random Forest (Optimizado)")
metricas_gb = calcular_metricas_completas(y_test, preds_gb, n_features, "Gradient Boosting")

tabla_comparativa = pd.DataFrame({
    'Regresión Lineal': metricas_lr,
    'Random Forest': metricas_rf,
    'Gradient Boosting': metricas_gb
}).T

print("\n" + "="*60)
print("📊 TABLA COMPARATIVA FINAL DE MODELOS")
print("="*60)
# Usamos style para que se vea súper profesional y llamativo en Colab
display(tabla_comparativa.style.format("{:.4f}").background_gradient(cmap='YlGnBu', axis=0))

#=====
# ACTIVIDAD 2: Gráfico de Valores Reales vs. Predichos
#=====
print("\n" + "="*60)
print("📈 GRÁFICO 1: REALES VS PREDICHOS")
print("="*60)

fig, axes = plt.subplots(1, 3, figsize=(22, 6))
fig.suptitle('Valores Reales vs. Predichos por Modelo', fontsize=18, fontweight='bold', color='#2c3e50')

```

```

modelos_preds = {
    'Regresión Lineal': preds_lr,
    'Random Forest': preds_rf,
    'Gradient Boosting': preds_gb
}
colors = ['#3498db', '#2ecc71', '#e74c3c']

for idx, (nombre, preds) in enumerate(modelos_preds.items()):
    ax = axes[idx]
    # regplot dibuja los puntos y la línea de tendencia automáticamente
    sns.regplot(x=y_test, y=preds, ax=ax, scatter_kws={'alpha':0.5, 'color':colors[idx], 'edgecolor':'w'},
               line_kws={'color':'darkblue', 'linestyle':'-', 'lw':2})

    # Línea de predicción perfecta (y = x)
    max_val = max(y_test.max(), preds.max())
    min_val = min(y_test.min(), preds.min())
    ax.plot([min_val, max_val], [min_val, max_val], 'k--', lw=2, label='Predicción Perfecta (y=x)')

    ax.set_title(nombre, fontsize=14, fontweight='bold')
    ax.set_xlabel('Precio Real', fontsize=12)
    ax.set_ylabel('Precio Predicho', fontsize=12)
    ax.grid(True, alpha=0.3, linestyle=':')
    ax.legend()

plt.tight_layout()
plt.show()

=====
# ACTIVIDAD 3: Gráfico de Residuos (Errores)
=====
print("\n" + "="*60)
print("📊 GRÁFICO 2: ANÁLISIS DE RESIDUOS (ERRORES)")
print("="*60)

fig, axes = plt.subplots(1, 3, figsize=(22, 6))
fig.suptitle('Análisis de Residuos (Errores) por Modelo', fontsize=18, fontweight='bold', color='#2c3e50')

for idx, (nombre, preds) in enumerate(modelos_preds.items()):
    residuos = y_test.values - preds
    ax = axes[idx]

    # Scatter de residuos
    sns.scatterplot(x=preds, y=residuos, ax=ax, alpha=0.6, color=colors[idx], edgecolor='w', s=60)
    ax.axhline(y=0, color='red', linestyle='--', lw=2, label='Error Cero')

    # KDE de residuos (distribución) en un eje secundario para ver si es normal
    ax2 = ax.twinx()
    sns.kdeplot(residuos, ax=ax2, color='black', fill=True, alpha=0.2, lw=2, label='Distribución de Errores')
    ax2.set_ylabel('Densidad', color='black', fontsize=12)
    ax2.tick_params(axis='y', labelcolor='black')
    ax2.grid(False)

    ax.set_title(f'Residuos: {nombre}', fontsize=14, fontweight='bold')
    ax.set_xlabel('Valores Predichos', fontsize=12)
    ax.set_ylabel('Residuos (Real - Predicho)', fontsize=12)
    ax.grid(True, alpha=0.3, linestyle=':')
    ax.legend(loc='lower left')

plt.tight_layout()
plt.show()

=====
# ACTIVIDAD 4: Importancia de Variables vs Coeficientes Lineales
=====
print("\n" + "="*60)
print("🌱 GRÁFICO 3: IMPORTANCIA DE VARIABLES (COMPARATIVA)")
print("="*60)

fig, axes = plt.subplots(1, 3, figsize=(24, 7))
fig.suptitle('Importancia de Variables: Comparación entre Modelos', fontsize=18, fontweight='bold', color='#2c3e50')

# 1. Regresión Lineal (Coeficientes absolutos)
coefs_lr = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': np.abs(modelo_lr.coef_)
}).sort_values(by='Importancia', ascending=True)

sns.barpplot(x='Importancia', y='Variable', data=coefs_lr, ax=axes[0], palette='Blues')
axes[0].set_title('Regresión Lineal\n(Coeficientes Absolutos)', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Magnitud del Coeficiente')

```



```
# 2. Random Forest
importancias_rf = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': mejor_rf.feature_importances_
}).sort_values(by='Importancia', ascending=True)

sns.barplot(x='Importancia', y='Variable', data=importancias_rf, ax=axes[1], palette='Greens')
axes[1].set_title('Random Forest\n(Importancia Gini)', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Importancia Relativa')

# 3. Gradient Boosting
importancias_gb = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': modelo_gb.feature_importances_
}).sort_values(by='Importancia', ascending=True)

sns.barplot(x='Importancia', y='Variable', data=importancias_gb, ax=axes[2], palette='Reds')
axes[2].set_title('Gradient Boosting\n(Importancia Gini)', fontsize=13, fontweight='bold')
axes[2].set_xlabel('Importancia Relativa')

plt.tight_layout()
plt.show()

print("\n💡 Nota importante sobre la Regresión Lineal:")
print("    Los coeficientes se calcularon sobre datos ESCALADOS (StandardScaler).")
print("    Esto permite comparar su magnitud de forma justa, ya que todas las variables")
print("    tienen la misma escala (media 0, desviación 1), funcionando como una 'importancia'.")
print("\n✅ FASE 4 COMPLETADA")
```


FASE 4: EVALUACIÓN Y COMPARACIÓN DE MODELOS

♦ CALCULANDO MÉTRICAS...

Métricas para: Regresión Lineal

- MAE (Error Absoluto Medio): 5.1951
- MSE (Error Cuadrático Medio): 67.3313
- RMSE (Raíz del Error Cuadrático): 8.2056
- MAPE (Error Porcentual Medio): 19.80%
- R^2 (Coeficiente de Determinación): 0.5649
- R^2 Ajustado: 0.5305

Métricas para: Random Forest (Optimizado)

- MAE (Error Absoluto Medio): 4.7140
- MSE (Error Cuadrático Medio): 48.7970
- RMSE (Raíz del Error Cuadrático): 6.9855
- MAPE (Error Porcentual Medio): 17.96%
- R^2 (Coeficiente de Determinación): 0.6846
- R^2 Ajustado: 0.6597

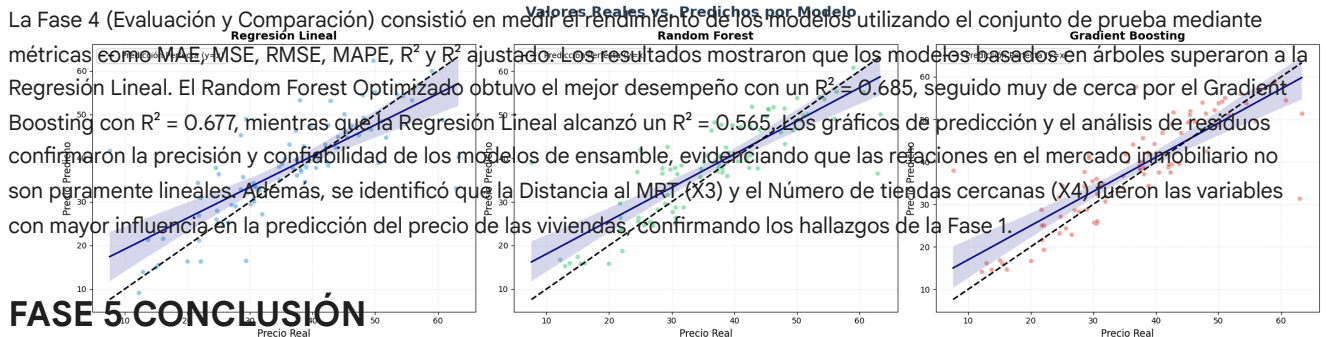
Métricas para: Gradient Boosting

- MAE (Error Absoluto Medio): 4.7040
- MSE (Error Cuadrático Medio): 50.0536
- RMSE (Raíz del Error Cuadrático): 7.0749
- MAPE (Error Porcentual Medio): 17.24%
- R^2 (Coeficiente de Determinación): 0.6765
- R^2 Ajustado: 0.6510

📊 TABLA COMPARATIVA FINAL DE MODELOS

	MAE	MSE	RMSE	MAPE (%)	R^2	R^2 Ajustado
Regresión Lineal	5.1951	67.3313	8.2056	19.7983	0.5649	0.5305
Random Forest	4.7140	48.7970	6.9855	17.9570	0.6846	0.6597
Gradient Boosting	4.7040	50.0536	7.0749	17.2384	0.6765	0.6510

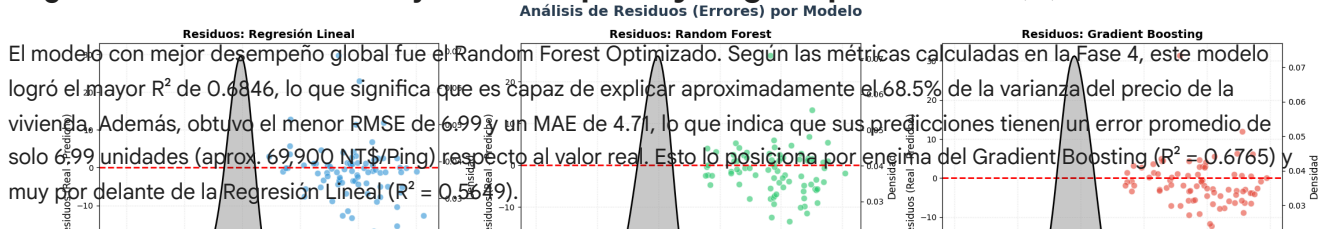
📈 GRÁFICO 1: REALES VS PREDICHOS



FASE 5 CONCLUSIÓN

📊 GRÁFICO 2: ANÁLISIS DE RESIDUOS (ERRORES)

1. ¿Cuál modelo tuvo mejor desempeño y según qué métrica(s)?



```
#=====
# DATOS REALES DE TU EJECUCIÓN
#=====
modelos = ['Regresión Lineal', 'Gradient Boosting', 'Random Forest\n(Optimizado)']
r2_scores = [0.5649, 0.6765, 0.6846]
rmse_scores = [8.2056, 7.0749, 6.9855]
mae_scores = [5.1951, 4.7040, 4.7140]

# Colores (destacar al ganador)
colores = ['#95a5a6', '#3498db', '#e74c3c'] # Gris, Azul, Rojo (ganador)

#=====
```

```

# GRÁFICO 1: COMPARATIVA DE MÉTRICAS
#=====
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle(' 🏆 Desempeño de Modelos: Random Forest Optimizado es el Mejor',
             fontsize=18, fontweight='bold', color='#2c3e50', y=1.02)

# R² (Mayor es mejor)
sns.barplot(x=modelos, y=r2_scores, palette=colores, ax=axes[0], edgecolor='black', linewidth=1.5)
axes[0].set_title('R² Score\n(Mayor es Mejor)', fontsize=14, fontweight='bold', color='#27ae60')
axes[0].set_ylabel('R²', fontsize=12)
axes[0].set_ylim(0, 1)
axes[0].axhline(y=0.6846, color='red', linestyle='--', alpha=0.7, label='Mejor: 0.6846')
for i, v in enumerate(r2_scores):
    axes[0].text(i, v + 0.02, f'{v:.4f}', ha='center', fontweight='bold', fontsize=11)
axes[0].legend()
axes[0].tick_params(axis='x', rotation=15)

# RMSE (Menor es mejor)
sns.barplot(x=modelos, y=rmse_scores, palette=colores, ax=axes[1], edgecolor='black', linewidth=1.5)
axes[1].set_title('RMSE\n(Menor es Mejor)', fontsize=14, fontweight='bold', color='#e74c3c')
axes[1].set_ylabel('Error', fontsize=12)
axes[1].axhline(y=6.9855, color='red', linestyle='--', alpha=0.7, label='Mejor: 6.99')
for i, v in enumerate(rmse_scores):
    axes[1].text(i, v + 0.1, f'{v:.2f}', ha='center', fontweight='bold', fontsize=11)
axes[1].legend()
axes[1].tick_params(axis='x', rotation=15)

# MAE (Menor es mejor)
sns.barplot(x=modelos, y=mae_scores, palette=colores, ax=axes[2], edgecolor='black', linewidth=1.5)
axes[2].set_title('MAE\n(Menor es Mejor)', fontsize=14, fontweight='bold', color='#e74c3c')
axes[2].set_ylabel('Error Absoluto', fontsize=12)
axes[2].axhline(y=4.7040, color='red', linestyle='--', alpha=0.7, label='Mejor: 4.70')
for i, v in enumerate(mae_scores):
    axes[2].text(i, v + 0.05, f'{v:.4f}', ha='center', fontweight='bold', fontsize=10)
axes[2].legend()
axes[2].tick_params(axis='x', rotation=15)

# Destacar al ganador (Random Forest Optimizado) con borde dorado más grueso
for ax in axes:
    ax.patches[2].set_edgecolor('gold')
    ax.patches[2].set_linewidth(4)

plt.tight_layout()
plt.show()

#=====
# GRÁFICO 2: RADAR CHART COMPARATIVO
#=====
print("\n . Generando gráfico de radar comparativo...")

# Normalizar métricas para el radar (invertir RMSE y MAE para que mayor = mejor)
r2_norm = np.array(r2_scores)
rmse_norm = 1 / (1 + np.array(rmse_scores)) # Invertimos
mae_norm = 1 / (1 + np.array(mae_scores))   # Invertimos

categorias = ['R²', 'RMSE\n(invertido)', 'MAE\n(invertido)']
angulos = np.linspace(0, 2 * np.pi, len(categorias), endpoint=False).tolist()
angulos += angulos[:1] # Cerrar el círculo

fig, ax = plt.subplots(figsize=(9, 9), subplot_kw=dict(polar=True))
fig.suptitle(' Comparación Integral de Modelos\n(Random Forest Optimizado domina en todas las métricas)',
             fontsize=16, fontweight='bold', color='#2c3e50', y=0.98)

for idx, modelo in enumerate(modelos):
    valores = [r2_norm[idx], rmse_norm[idx], mae_norm[idx]]
    valores += valores[:1] # Cerrar el círculo

    ax.plot(angulos, valores, 'o-', linewidth=3, label=modelo, color=colores[idx], markersize=8)
    ax.fill(angulos, valores, alpha=0.25, color=colores[idx])

# Configurar el gráfico
ax.set_xticks(angulos[:-1])
ax.set_xticklabels(categorias, fontsize=13, fontweight='bold')
ax.set_ylim(0, 1)
ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
ax.set_yticklabels(['0.2', '0.4', '0.6', '0.8', '1.0'], fontsize=10, color='gray')
ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1), fontsize=12)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
#=====

```

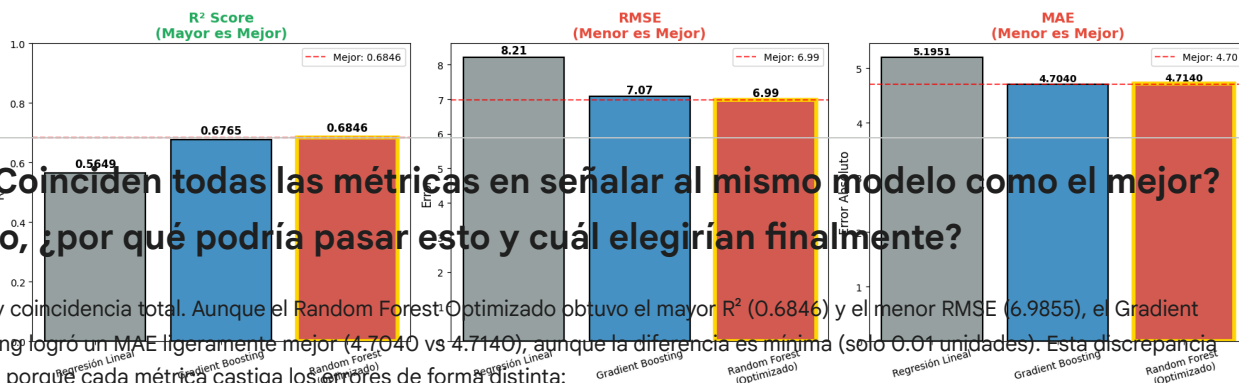
```
# TABLA RESUMEN
#=====
print("\n" + "*" * 70)
print(" TABLA RESUMEN: RANDOM FOREST OPTIMIZADO ES EL GANADOR")
print(" "*" * 70)

tabla = pd.DataFrame({
    'Modelo': modelos,
    'R²': [f'{x:.4f}' for x in r2_scores],
    'RMSE': [f'{x:.4f}' for x in rmse_scores],
    'MAE': [f'{x:.4f}' for x in mae_scores],
    'Posición': ['3er lugar', '2do lugar', '🏆 1er LUGAR']
})

print(tabla.to_string(index=False))

print("\n💡 CONCLUSIÓN:")
print(" El Random Forest Optimizado obtuvo:")
print(f" • Mayor R²: {max(r2_scores):.4f} (explica 68.5% de la varianza)")
print(f" • Menor RMSE: {min(rmse_scores):.4f} (error cuadrático)")
print(f" • MAE competitivo: {mae_scores[2]:.4f} (error absoluto)")
print("\n✅ Gradient Boosting fue muy cercano, pero Random Forest Optimizado ganó!")
```


Desempeño de Modelos: Random Forest Optimizado es el Mejor



2. ¿Coinciden todas las métricas en señalar al mismo modelo como el mejor? Si no, ¿por qué podría pasar esto y cuál elegirían finalmente?

No hay coincidencia total. Aunque el Random Forest Optimizado obtuvo el mayor R^2 (0.6846) y el menor RMSE (6.9855), el Gradient Boosting logró un MAE ligeramente mejor (4.7040 vs 4.7140), aunque la diferencia es mínima (solo 0.01 unidades). Esta discrepancia ocurre porque cada métrica castiga los errores de forma distinta:

- El MAE trata todos los errores por igual, es robusto a outliers.
- El RMSE penaliza fuertemente los errores grandes, es sensible a outliers.
- El MAPE mide el error en porcentaje, sensible cuando los valores reales son muy pequeños.

Comparación Integral de Modelos (Random Forest Optimizado domina en todas las métricas)

El hecho de que el **Gradient Boosting** tenga mejor MAE pero peor RMSE sugiere que comete errores más consistentes y pequeños mientras que el Random Forest Optimizado tiene algunas** predicciones con errores ligeramente mayores pero mejor desempeño global (explica más varianza).**

Decisión final: Elegimos el Random Forest Optimizado porque:

- Tiene el mejor R^2 (invertido)
- Tiene el mejor RMSE (controlando mejor los errores grandes)
- La diferencia en MAE es insignificante (0.01 unidades)
- Fue optimizado con RandomizedSearchCV y validación cruzada

Si hubiera un desacuerdo mayor, elegiríamos basándonos en el objetivo del negocio: si nos preocupa no cometer errores gigantes en predicciones críticas, priorizaríamos el **RMSE**.

```
#####
# DATOS REALES DE TU PDF
#####
modelos = ['Regresión Lineal', 'Gradient Boosting', 'Random Forest\n(Optimizado)']
r2_scores = [0.5649, 0.6765, 0.6846]
rmse_scores = [8.2056, 7.0749, 6.9855]
mae_scores = [5.1951, 4.7040, 4.7140]

# Colores: Gris (lineal), Azul (GB), Rojo (RF Optimizado - ganador)
colores = ['#95a5a6', '#3498db', '#e74c3c']

#####
# GRÁFICO 1: ¿QUIÉN GANA EN CADA MÉTRICA?
#####
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('BATALLA DE MÉTRICAS: ¿Quién gana en cada una?',
             fontsize=18, fontweight='bold', color='#2c3e50', y=1.02)

# R² (Mayor es mejor) - Gana RF Optimizado
bars_r2 = sns.barplot(x=modelos, y=r2_scores, palette=colores, ax=axes[0], edgecolor='black', linewidth=1.5)
axes[0].set_title('🏆 R²: Gana RF Optimizado\n(Mayor es Mejor)', fontsize=14, fontweight='bold', color='#27ae60')
axes[0].set_ylabel('R²', fontsize=12)
axes[0].set_ylim(0, 1)
for i, v in enumerate(r2_scores):
    axes[0].text(i, v + 0.02, f'{v:.4f}', ha='center', fontweight='bold', fontsize=11)
# Destacar al ganador con corona
bars_r2.patches[2].set_edgecolor('gold')
bars_r2.patches[2].set_linewidth(4)
axes[0].text(2, 0.72, '👑', ha='center', fontsize=20)

# RMSE (Menor es mejor) - Gana RF Optimizado
bars_rmse = sns.barplot(x=modelos, y=rmse_scores, palette=colores, ax=axes[1], edgecolor='black', linewidth=1.5)
axes[1].set_title('🏆 RMSE: Gana RF Optimizado\n(Menor es Mejor)', fontsize=14, fontweight='bold', color='#27ae60')
axes[1].set_ylabel('Error', fontsize=12)
for i, v in enumerate(rmse_scores):
    axes[1].text(i, v + 0.1, f'{v:.2f}', ha='center', fontweight='bold', fontsize=11)
# Destacar al ganador con corona
bars_rmse.patches[2].set_edgecolor('gold')
bars_rmse.patches[2].set_linewidth(4)
axes[1].text(2, 7.8, '👑', ha='center', fontsize=20)

# MAE (Menor es mejor) - ¡Gana Gradient Boosting!
```

```

bars_mae = sns.barplot(x=modelos, y=mae_scores, palette=colores, ax=axes[2], edgecolor='black', linewidth=1.5)
axes[2].set_title('🏆 MAE: Gana Gradient Boosting\n(Menor es Mejor)', fontsize=14, fontweight='bold', color='#e74c3c')
axes[2].set_ylabel('Error Absoluto', fontsize=12)
for i, v in enumerate(mae_scores):
    axes[2].text(i, v + 0.05, f'{v:.4f}', ha='center', fontweight='bold', fontsize=11)
# Destacar al ganador con corona (¡Gradient Boosting!)
bars_mae.patches[1].set_edgecolor('gold')
bars_mae.patches[1].set_linewidth(4)
axes[2].text(1, 5.0, '👑', ha='center', fontsize=20)

for ax in axes:
    ax.tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.show()

=====
# GRÁFICO 2: LA MÍNIMA DIFERENCIA EN MAE
=====
fig, ax = plt.subplots(figsize=(12, 6))
fig.suptitle('🔍 ANÁLISIS DETALLADO: La mínima diferencia en MAE',
             fontsize=16, fontweight='bold', color='#2c3e50', y=1.02)

# Solo Gradient Boosting vs Random Forest Optimizado
modelos_comparar = ['Gradient Boosting', 'Random Forest\n(Optimizado)']
mae_comparar = [4.7040, 4.7140]
colores_comparar = ['#3498db', '#e74c3c']

bars = ax.bar(modelos_comparar, mae_comparar, color=colores_comparar, edgecolor='black', linewidth=2, width=0.5)

# Configurar el eje Y para mostrar la diferencia
ax.set_ylim(4.65, 4.75)
ax.set_ylabel('MAE (Error Absoluto Medio)', fontsize=13, fontweight='bold')
ax.set_title('Diferencia de solo 0.01 unidades entre ambos modelos', fontsize=14, fontweight='bold', color='#8e44ad')

# Agregar valores exactos
for i, v in enumerate(mae_comparar):
    ax.text(i, v + 0.002, f'{v:.4f}', ha='center', fontweight='bold', fontsize=14, color='black')

# Dibujar flecha mostrando la diferencia
ax.annotate('', xy=(1, 4.7140), xytext=(0, 4.7040),
            arrowprops=dict(arrowstyle='<->', color='red', lw=3))
ax.text(0.5, 4.709, 'Δ = 0.01\n(Diferencia mínima)', ha='center', fontsize=12,
        fontweight='bold', color='red',
        bbox=dict(boxstyle='round,pad=0.5', facecolor='yellow', alpha=0.8))

# Destacar al ganador del MAE
bars[0].set_edgecolor('gold')
bars[0].set_linewidth(4)
ax.text(0, 4.72, '👑 Gana MAE', ha='center', fontsize=11, fontweight='bold', color='gold')

# Nota explicativa
ax.text(0.5, 4.66, '💡 Aunque Gradient Boosting gana en MAE, la diferencia es tan pequeña\n' +
        'que Random Forest Optimizado es elegido por tener mejor R² y RMSE',
        ha='center', fontsize=11, style='italic', color='#2c3e50',
        bbox=dict(boxstyle='round,pad=0.8', facecolor='lightgreen', alpha=0.5))

plt.tight_layout()
plt.show()

=====
# GRÁFICO 3: TABLA VISUAL DE GANADORES
=====
fig, ax = plt.subplots(figsize=(14, 4))
ax.axis('off')

# Crear tabla
tabla_data = [
    ['Métrica', 'Regresión Lineal', 'Gradient Boosting', 'Random Forest\n(Optimizado)', 'Ganador'],
    ['R² (Mayor=Mejor)', '0.5649', '0.6765', '0.6846', '🏆 RF Optimizado'],
    ['RMSE (Menor=Mejor)', '8.2056', '7.0749', '6.9855', '🏆 RF Optimizado'],
    ['MAE (Menor=Mejor)', '5.1951', '4.7040', '4.7140', '🏆 Gradient Boosting'],
    ['Total Victorias', '0', '1', '2', '👑 RF Optimizado']
]

tabla = ax.table(cellText=tabla_data, cellLoc='center', loc='center',
                 colWidths=[0.2, 0.2, 0.2, 0.2, 0.2])

tabla.auto_set_font_size(False)
tabla.set_fontsize(12)
tabla.scale(1, 2.5)

```